

FluxLang

Linguagem de automação e workflows - conceito inicial da v1

Ideia central A FluxLang é uma linguagem tipada voltada à automação de tarefas. O programa é organizado em workflows reutilizáveis, coordenados por uma função main, com suporte a execução declarativa de comandos, controle de fluxo, listas, objetos e tratamento explícito de erros.

VISÃO RÁPIDA

Propósito	Automatizar tarefas e organizar fluxos de execução.
Estrutura	main + funções comuns + workflows.
Foco da v1	Execução de comandos; HTTP e recursos mais avançados ficam para versões futuras.

1. Estrutura da linguagem

- O arquivo pode conter var, const, func e workflow em qualquer ordem.
- Deve existir exatamente uma func main () como ponto de entrada.
- Workflows são unidades de automação e são chamados explicitamente com call.
- Funções comuns têm no máximo um valor de retorno; workflows trabalham com (resultado, erro).

```
const version:string = "1.0";

workflow build(project:string) {
    // automação
}

func main () {
    result, err = call build("backend");
    abort if err;
}
```

2. Tipos, variáveis e estruturas de dados

- Tipos básicos: string, int, float, bool, object e list<tipo>.
- null pode representar ausência de valor.
- var declara valores mutáveis; const exige valor inicial e impede reatribuição da variável.
- Declarações e atribuições múltiplas são permitidas.

```
var nome, telefone:string = "Andrew", "99999";
var idade:int;
const version:string = "1.0";

nome, telefone = "Carlos", "88888";
```

Listas são homogêneas e suportam índice, length, push e remove.

```
var nomes:list<string> = ["Ana", "Bruno"];
nomes[0] = "Andrew";
nomes.push("Carlos");
nomes.remove(1);

for nome in nomes {
    terminal.log(nome);
}
```

Objects agrupam dados por campos e usam acesso por ponto.

```
var user:object = {
    name: "Andrew",
    age: 17,
    active: true,
};

user.name = "Carlos";
```

3. Controle de fluxo e expressões

- Condicional: `if (...) { ... } else { ... }`. O `else` é opcional.
- Repetição: `while (...)` e `for item in lista`.
- Laços suportam `break`; e `continue`;
- Operadores: `!`, `*`, `/`, `%`, `+`, `-`, comparações, `==`, `!=`, `&&` e `||`, com precedência tradicional.
- O operador `+` também poderá ser usado sintaticamente para concatenação de strings.

```
if (idade >= 18) {
    terminal.info("Maior de idade");
} else {
    terminal.warn("Menor de idade");
}

while (ativo) {
    break;
}
```

4. Terminal e conversões

- `terminal` concentra entrada e saída: `log`, `info`, `warn`, `error` e `input`.
- `terminal.input(...)` sempre retorna string.
- Conversões explícitas: `toString`, `toInt`, `toFloat` e `toBool`.

```
terminal.info("Iniciando processo");
var nome:string = terminal.input("Nome: ");
var idade:int = toInt(terminal.input("Idade: "));
terminal.log("Olá " + nome);
```

5. Workflows e tratamento de erros

- Workflows podem receber parâmetros e chamar outros workflows com `call`.
- O retorno conceitual de workflow é (resultado, erro); o resultado pode ser `null`.
- Se um workflow termina naturalmente, o retorno implícito é (`null`, `null`).
- `stop if erro`; encerra o workflow atual e propaga o erro.
- `abort if erro`; encerra a execução global.

- continue if erro; reconhece o erro e segue o fluxo.

```
workflow build(project:string) {
  output, err = run {
    command: "npm",
    flags: ["run", "build"],
    workingDir: project,
  };

  if (err != null) {
    terminal.error(err);
    stop if err;
  }

  return output, null;
}
```

6. Automação com run

- run usa sintaxe declarativa em bloco.
- command é obrigatório; flags e workingDir são opcionais.
- flags é uma list<string>. Os campos podem aparecer em qualquer ordem.
- O bloco run usado em uma atribuição termina com ;.

```
result, err = run {
  command: "npm",
  flags: ["test", "--watch"],
  workingDir: "./backend",
};
```

7. Regras de sintaxe já definidas

- A linguagem é case-sensitive.
- Identificadores começam com letra ou _ e podem continuar com letras, números ou _.
- Strings usam apenas aspas duplas e suportam escapes básicos como \n, \t, \" e \\.
- Números usam formato decimal simples; floats exigem dígitos dos dois lados do ponto.
- O sinal - é tokenizado separadamente e pode funcionar como operador unário.
- Comentários: // linha e /* bloco */.
- Whitespace é ignorado fora de strings e comentários.
- Instruções simples terminam com ;. Blocos não precisam de ;, exceto quando fazem parte de expressão como run.

8. Exemplo integrado

```
const projectDir:string = "./backend";

workflow testProject(project:string) {
  output, err = run {
    command: "npm",
    flags: ["test"],
    workingDir: project,
  };

  if (err != null) {
    terminal.error(err);
    stop if err;
  }

  terminal.info("Testes executados");
  return output, null;
}

workflow buildProject(project:string) {
  output, err = run {
    command: "npm",
    flags: ["run", "build"],
    workingDir: project,
  };

  stop if err;
  return output, null;
}

func main () {
  testResult, testErr = call testProject(projectDir);
  abort if testErr;

  buildResult, buildErr = call buildProject(projectDir);
  abort if buildErr;

  terminal.info("Workflow finalizado com sucesso");
}
```

Escopo da v1 A primeira versão prioriza o núcleo necessário para lexer, parser e AST. Requisições HTTP, recursão de workflows e outros recursos avançados ficam para versões posteriores.